AgentCore as the Sandbox in Agentic RL

AgentCore Runtime as the execution sandbox, integrated with slime

Kaige Yang — Senior Applied Scientist, AWS

September 10, 2026

Outline

Section 1

1. Why sandboxes — what frontier labs run in them

Purpose & roadmap — AgentCore as the sandbox for agentic RL

One idea: in agentic RL the policy acts over many turns against an executable environment, and a **programmatic checker** — not an LLM judge — turns the outcome into a scalar reward (RLVR).

Roadmap (6 sections):

1. **Why sandboxes** — what frontier labs run in sandboxes, and how they score it.
2. **Architecture & pipeline** — reference architecture, the five components, slime + its async rollout.
3. **Foundations of AgentCore** — Runtime / session / microVM, CPU vs GPU compute, session lifecycle & launch/terminate.
4. **Integrating into slime** — the call path, one hook, the episode loop, and the key code.
5. **The demos** — all 21 at a glance, each with its real use case.
6. **Special demos** — the TITO mechanism, scaling (harness × inference × sandbox), mixed-task, GPU sharing, image pool, real-CLI branching, heartbeat.

*Scope: AgentCore = the execution **sandbox**; **slime** = the trainer; the **harness stays in-process**.*

What tasks do frontier labs run in sandboxes

Untrusted output runs in an isolated sandbox; a **programmatic verifier** produces the reward. This is the dominant recipe in recent tech reports — Kimi K2 [5] reports **>10,000 concurrent sandboxes** for agentic tool-use / code RL.

Two verifier styles: *verifiable / rule-based* (binary, ungameable) vs *reference-based judge* (open-ended). This project uses **RLVR only**.

Model	Verifier	Tasks	Reward
DeepSeek-R1 [2]	rule verifier + compiler	math, competitive code	tests pass / match
Qwen3 [3]	RLVR + live tool env	math, code, tool-use, RAG	rule + judge
Llama-Nemotron [4]	LLM-judge	science, math, instruction	match + format
Kimi K2 [5]	simulator + real exec sandboxes	agentic tool use, code, SWE	tests + checks

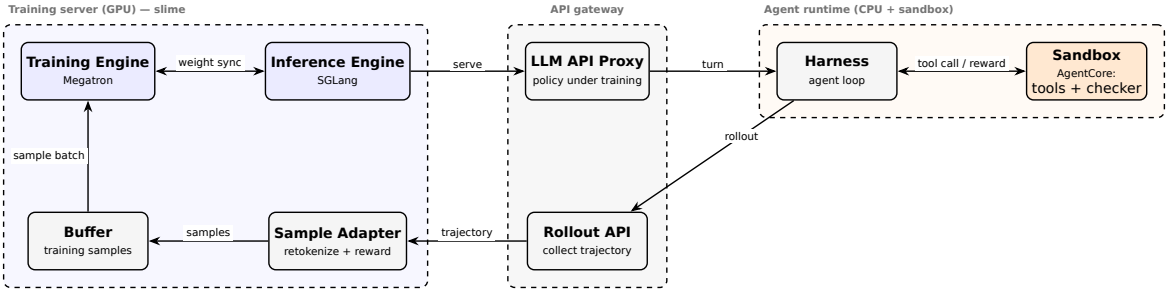
What tasks do frontier labs run in sandboxes (cont.)

Same recipe throughout this deck: ***one isolated sandbox per episode****, a* ***programmatic checker*** *for the reward.*

Section 2

2. Agentic RL architecture & the training pipeline

The reference architecture — one diagram



Generation lane (top, →); training-data lane (bottom, ←); disaggregated design after Agent Lightning [1], slime [9] as trainer. ***AgentCore fills the sandbox seam.***

The pipeline — five components, one job each

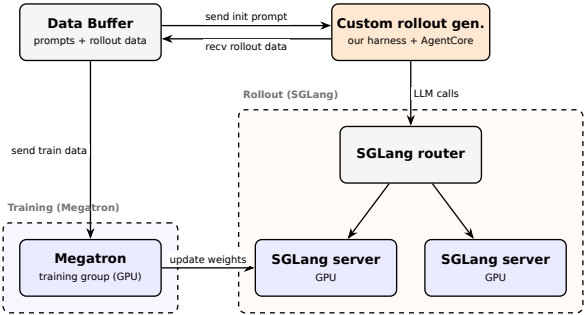
- **Trainer / Inference** (GPU) — learn, and generate turns at the *current* policy (kept fresh by **weight sync**).
- **Proxy** (CPU) — makes every model call hit the *policy under training*, not an external API.
- **Harness** (CPU) — the agent loop that owns one episode end-to-end.
- **Sandbox** (CPU/GPU) — the isolated box untrusted output runs in; the **checker scores the reward**.

Two lanes, one cycle:

- **Generation lane (→):** trainer weights → SGLang → proxy → harness turn → sandbox tool call → observation → next turn.
- **Training-data lane (←):** trajectory → reward attached → loss-masked sample → buffer → trainer step → weight sync.

AgentCore fills the sandbox seam — ad-hoc + isolated per episode. The harness needs *warmth*, not isolation, so it stays **in-process**.

slime — the trainer, and its asynchronous rollout



slime [9] = Megatron (training) + SGLang [27] (rollout), bridged by a **Data Buffer**; the agentic loop plugs into its rollout **generation hook** — no fork of the training kernel.

- **Sync mode** alternates train / rollout — each side's GPUs idle while the other runs.
- **Async driver** (`train_async.py`) overlaps them: Megatron trains batch k while the rollout side already generates batch $k{+}1$.
- **Fully-async rollout** (`fully_async_rollout`) keeps a *fixed pool of in-flight episodes across rollout boundaries*, so a step never waits on the slowest trajectory — the **long-tail** problem of agentic RL.

slime — the trainer, and its asynchronous rollout (cont.)

- **Why it matters here:** AgentCore sandboxes have **variable, sometimes long** per-episode latency; async rollout keeps the trainer fed while they run — at the cost of **staleness** (rollouts generated on an older policy version). This project runs the fully-async path (`MODE=B`).

3. Foundations of AgentCore Runtime

AgentCore basics — three concepts, precisely

- **Runtime** — a versioned, containerized host behind an ARN (`CreateAgentRuntime` [D9]). The deployable unit; holds no per-user state.
- **Session** — one episode, addressed by a `runtimeSessionId`. First invoke cold-starts compute; the same id re-routes to that warm compute [D3].
- **microVM** — the unit of isolation: own kernel, CPU, memory, filesystem; sanitized on termination [D2].

Cardinality is 1 : N : 1 — one Runtime serves many sessions; each session = exactly one microVM. **The number of Runtimes tracks distinct sandbox *images*, not tasks.**

Concurrency — episodes, sessions & the Runtime together

`N` concurrent episodes run as `N` session ids on ONE Runtime — not `N` Runtimes.

- The `generate` hook opens **one session per episode** (`CONC` in flight at once); each session id pins that episode to **its own microVM** (or its own GPU instance).
- Every tool op of an episode carries the *same* `runtimeSessionId`, so it returns to the **same warm microVM** — state persists within the episode, isolation holds across episodes.
- The capacity provider places those `N` sessions for you; **one image (one ARN)** backs them all. Add a **second ARN** only for a different image/substrate (e.g. GPU), then round-robin the pool.

So concurrency = more sessions, not more Runtimes.

Why a microVM (not just a container)

Running **untrusted, model-generated code** — isolation is the whole point: one episode's code can't see another's files, the trainer's weights, or the host, and the box is wiped at session end.

	Docker container	**AgentCore microVM**	E2B
Boundary	shared host kernel	**own kernel** (Firecracker-class)	own kernel
Runs your image?	yes	**your OCI image in a microVM**	Dockerfile templates
Managed by	you	**AWS, in your account** (IAM/VPC)	E2B service
GPU	host-dependent	**via Instances**	not core

Two compute types — CPU microVM vs GPU Instances

CPU work belongs on the microVM; Instances exist to add a GPU [D5] (EC2 in your account via a capacity provider). **No cross-session GPU time-slicing** — scale by adding sessions/instances, not by fractioning one (no MIG on L4).

Property	Serverless microVM	Instances (GPU)
GPU	none	g4dn/g5/g6/g6e… + inf2
Max session	8 h	14 d
Isolation	1 microVM / session	1 instance / session (**1 GPU**)
Filesystem	ephemeral	persistent EBS
Agents / session	1	many

Two compute types — CPU microVM vs GPU Instances (cont.)

One session = one whole instance (session→instance is 1:1 — you get the *instance*, not a GPU). Two ids acquire two *separate* instances, so you can't split one multi-GPU box across sessions. To use every GPU of one instance: **one shared session + a self-managed in-instance Ray farm** (`demo_gpu_shared` / `demo_gpu_local`).

The microVM’s resource envelope — three limits that bite

Serverless isolation is cheap, but the microVM is **small and ephemeral**. Three limits shape every image:

Limit	Value	Why it bites	Work-around
Image size	**≤ ~2 GB** (arm64, project-measured)	a CUDA/torch wheel alone is ~2.7 GB → build fails	install heavy wheels **at `CMD`**, not baked (GPU image: ~90 s cold)
Disk	**ephemeral only** — no persistent volume [D3]	a `git cloned` repo / installed pkg is **wiped at session end**	durable state → S3 / session Memory, or move to **Instances** (EBS) [D5]
vCPU / RAM	**metered, not sized** (vCPU-hr + GB-hr) [D13] — no fixed knob	a data-heavy checker can exceed the account quota	check **Service Quotas**; big fixed footprint → **Instances** (EC2 spec)

The microVM’s resource envelope — three limits that bite (cont.)

- **Example:** the g6 sandbox image can’t bake torch (>2 GB) → `pip install torch` at `CMD` so the wheel lands in the ephemeral workspace at boot; a big SWE repo + persistent workspace → move to **Instances** (EBS, 14 d).
- **Rule of thumb:** *files-in-a-session* for light deps → *`CMD`-time install* for medium → *Instances* when the footprint is large and **fixed**.

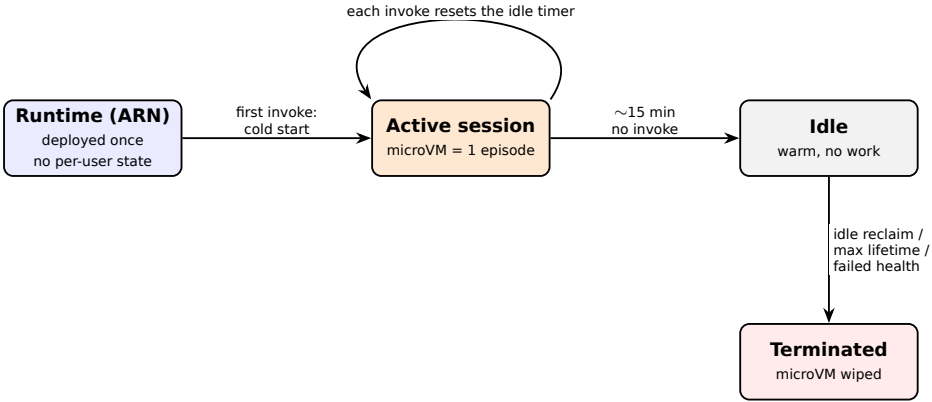
Building & deploying a Runtime — three ways, one contract

Contract [D17]: ARM64 image, serve `0.0.0.0:8080` with `/invocations` + `/ping`. Three ways to put your code on a Runtime (increasing control): the **`@aws/agentcore` CLI** [D16] (zero-Docker zip) → the **`bedrock-agentcore` SDK** [D15] (`@app.entrypoint` implements the contract) → **bring-your-own container** (max control — this project).

```
FROM public.ecr.aws/docker/library/python:3.12-slim   # CPU sandbox
RUN pip install --no-cache-dir bedrock-agentcore
COPY agentcore_slime_demo/sandbox_server_agent.py /app/
CMD ["python","-m","agentcore_slime_demo.sandbox_server_agent"]  # serves :8080
```

- **GPU image:** point `LD_LIBRARY_PATH=/usr/lib64` at the injected `libcuda.so.1`, and `pip install torch` **at `CMD`** (not baked) to stay under the **2048 MB** cap.
- **Deploy:** ECR create → `buildx --platform` (arm64 microVM / amd64 GPU) → push → `CreateAgentRuntime` → (GPU) `CreateCapacityProvider` (`g6.xlarge`, VPC, `idleInstanceTimeout=600`) + attach.

Session lifecycle — launch, idle timer, terminate



*Launch is **implicit** (first invoke); terminate is **implicit** (idle / max-lifetime / health). A heartbeat ping resets the timer (`demo_cpu_quiesce`).*

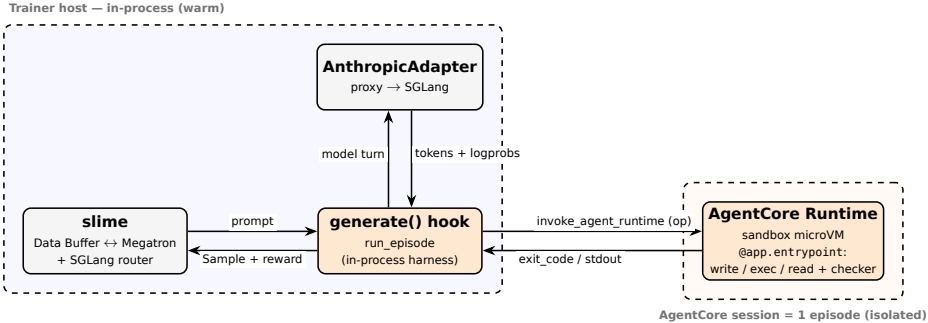
How to launch & terminate a session — the mechanics

- **Launch = the first `invoke_agent_runtime` on a fresh `runtimeSessionId`.** There is no explicit “start” call; the id (≥33 chars) *is* the launch. The first op cold-starts the microVM (or GPU instance — minutes on a cold GPU); every later op with that id re-routes to the same warm box.
- **Terminate = passive.** No explicit stop on the happy path — the session ends on **idle reclaim (~15 min)**, **max lifetime** (8 h microVM / 14 d instance), or **failed health**. Each invoke resets the idle timer.
- **Consequence in the RL loop:** a **100-row dataset does not launch 100 instances** — only `CONC` episodes are in flight, recycling through rows over steps. **Concurrency multiplies instances, not data.**
- The **weight-sync pause can trip the idle clock** and silently lose a warm session → the resilience levers in §5.
- **Latency: measure, don’t assume** — every op is instrumented (`sandbox_ms`, `sglang_gen_ms`).

Section 4

4. Integrating AgentCore into slime as the sandbox

How AgentCore sessions integrate into slime — the call path



*One `generate()` hook, one **`runtimeSessionId` per episode**; each tool call = one SigV4 `invoke_agent_runtime`.*

Plugging into slime — one hook, no fork

slime exposes the rollout as a **custom generation function**, so the agentic workload plugs in without touching the training kernel:

- `--custom-generate-function-path ⋯generate.generate` — drives the in-process harness; each tool call → one `invoke_agent_runtime` keyed by that episode's **`runtimeSessionId`**.
- **Design choice:** AgentCore is the **sandbox, not the harness**. The harness stays in-process (warm); the sandbox is the isolated per-session microVM. (It *can* host the harness too — `demo_cpu_branching` runs the real Claude Code CLI inside a Runtime.)
- The returned **loss-masked `Sample` + reward** flows straight back into slime's buffer — no fork of Megatron/SGLang.

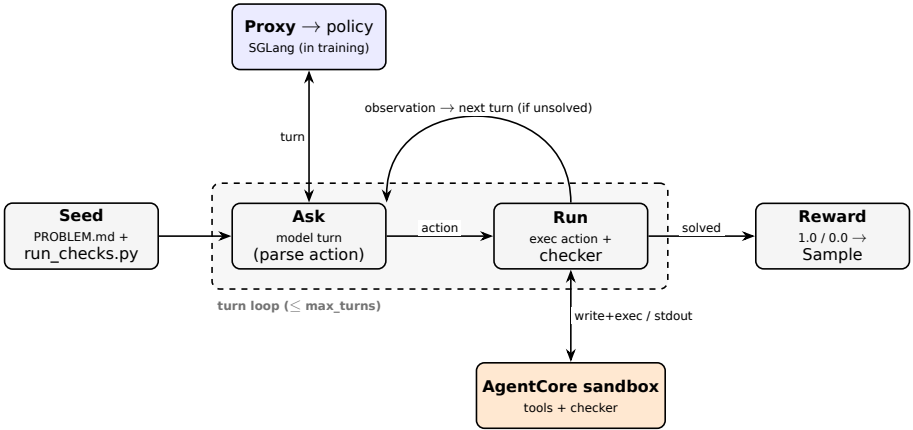
Who owns the lifecycle

slime's training kernel controls **neither** the Runtime nor the session.

- **Runtime** (create/delete the ARN) — the **deploy driver**, once before training. slime just gets the ARN.
- **Session** (per episode) — the `generate` hook's sandbox client: **launch implicit** (first invoke cold-starts the microVM); **terminate implicit** (idle reclaim, no explicit stop on the happy path).

Because termination is *passive*, the weight-sync pause can trip the idle clock and lose a warm session — the failure mode the resilience demo (`cpu_quiesce`) addresses.

The episode, end to end — the loop



Seed → Ask (proxy) → Run (sandbox + checker) → Reward; unsolved loops back.

Step-by-step — one MBPP episode

1. **Deploy (once):** build the arm64 sandbox image → `CreateAgentRuntime` → get the ARN (the deploy driver, not slime).
2. **Launch run:** `run.sh` sets `SANDBOX=agentcore`, `FAMILY`, `CONC`, `EPISODE_MAX_TURNS`, then `ray job submit ⋯ train_async.py --custom-generate-function-path ⋯generate.generate`.
3. **Open session:** slime hands a prompt to `generate()`, which rebuilds the `Task` and calls `make_sandbox("agentcore")` → **cold-starts** that episode's microVM (`runtimeSessionId`).
4. **Seed:** `write_file PROBLEM.md` + hidden `run_checks.py` into the session.
5. **Turn loop (≤ `EPISODE_MAX_TURNS`):** `policy.complete` via the **proxy** (TITO tokens) → extract `solution.py` → `exec("python3 run_checks.py")` in the sandbox → on failure, feed stdout back.
6. **Reward & learn:** `exit_code == 0` → **reward 1.0/0.0** from the in-sandbox checker → `build_token_stream` → loss-masked `Sample` → buffer → Megatron step → **weight sync** to SGLang.

Key code (1/2) — the in-process generate hook

slime calls this once per prompt: build the policy at the SGLang router, open **one sandbox per episode**, run the loop, return one loss-masked `Sample`.

```python
# agentcore_slime_demo/generate.py   (--custom-generate-function-path)
async def generate(args, sample, sampling_params, evaluation=False):
    policy  = SGLangPolicy(router_url=f"http://{args.sglang_router_ip}:{args.sglang_router_port}",
                           model_path=args.hf_checkpoint)
    task    = _task_from_sample(sample)            # rebuild Task from row metadata
    sandbox = make_sandbox(os.environ.get("SANDBOX", "agentcore"))
    async with sandbox as sb:                      # one session == one episode
        traj = await run_episode(policy, sb, task, sampling_params)
    tokens, resp_len, loss_mask, logprobs = build_token_stream(traj, tokenizer)
    return Sample(tokens=tokens, response_length=resp_len, reward=traj.reward,
                  loss_mask=loss_mask, rollout_log_probs=logprobs)
```

- Loss mask = **1 on the policy's own tokens, 0 on prompt / observation / tool tokens**.
- `build_token_stream` reuses the **recorded token_ids per turn** — no re-tokenization (TITO); returns a *bare* `Sample` (slime's async path chokes on nested lists).

Key code (2/2) — the sandbox client & the Runtime server

Each `write_file` / `exec` / `read_file` = **one SigV4 `invoke_agent_runtime`** [D8] keyed by the episode's session id; boto3 offloaded with `asyncio.to_thread` so `N` episodes run concurrently (with retries for transient blips + the GPU cold launch).

```python
# client — agentcore_runtime_sandbox.py
def _invoke_sync(self, payload):
    resp = self._client.invoke_agent_runtime(agentRuntimeArn=self._arn,
        runtimeSessionId=self._session_id,        # >=33 chars; one per episode
        payload=json.dumps(payload).encode())
    return json.loads(resp["response"].read().decode())

# server (deployed image) — the checker runs HERE, so the microVM is the reward source
app = BedrockAgentCoreApp()
@app.entrypoint
def entrypoint(request):        # dispatch on op: ping / write_file / read_file / exec
    return handle(request)
app.run()                       # serves 0.0.0.0:8080  (/invocations + /ping)
```

CPU → GPU is one env var (`SANDBOX=`) — zero change to the loop.

One image, many tasks & dependencies

One microVM = one image per session. Task diversity is absorbed *inside* one generic image, in increasing order of how structural the difference is:

1. **Files, not images** — most CPU families (math `sympy`, code `unittest`, SQL `sqlite3`, tool-use, light CTF) share **one generic Python image**; prompt, seed files, and hidden checker are written into *that episode's session*.
2. **Per-session install** — heavier deps go into the ephemeral workspace: **`CMD`-time** (the GPU image installs the ~2.7 GB torch wheel at boot to stay under the ~2 GB arm64 cap) or **seed-time** `git clone`/`pip`.
3. **Per-capability image** — only a genuine substrate difference (chiefly the **GPU** toolchain: `nvcc` + a real GPU, or a prebuilt dependency set) needs a **second Runtime ARN** (`demo_cpu_imagepool`, the GPU demos).

Section 5

5. The demos

The demo set — 21 demos, one design

Policy always **Qwen3-4B**, harness **in-process** (except `cpu_branching`), AgentCore the **sandbox**; only the *substrate* and *task* vary.

- **6 canonical** — `cpu_single` (MBPP), `cpu_math`, `gpu_single` (KernelBench), `gpu_shared` (Ray farm), `cpu_quiesce` + `gpu_snapshot` (resilience).
- **8 per-task CPU fan-out** — one real HF dataset each: gsm8k, math500, deepscaler, humaneval, gretelsql, hotpotqa, xlam, mcp.
- **2 scale a new dimension** — `harness_scale`, `mixtask`.
- **5 substrate/design variants** (newest) — `cpu_branching`, `cpu_imagepool`, `cpu_customruntime`, `gpu_customruntime`, `gpu_local`.

Honest status: only `cpu_single` and `gpu_single` carry ~10-step curves; the rest are NR=3 smoke or ready-to-run recipes.

Demos (1/3) — canonical + resilience

Demo	Substrate	Task	Real use case	Status
`cpu_single`	CPU µVM	MBPP repair	test-driven coding	**trained** (10-step)
`cpu_math`	CPU µVM	math words	math post-training	smoke
`gpu_single`	GPU g6	KernelBench	CUDA kernel agent	**trained** (10-step)
`gpu_shared`	GPU g6 farm	KernelBench	GPU packing (scarce GPUs)	smoke green (local GPU)
`cpu_quiesce`	CPU µVM	heartbeat	keep sandbox warm over sync	offline
`gpu_snapshot`	GPU inst.	snapshot	survive instance recycle	offline + real recycle

Demos (2/3) — per-task CPU fan-out (real HF datasets)

Demo	Substrate	Task	Real use case	Status
`cpu_gsm8k`	CPU µVM	GSM8K	grade-school math	smoke, eval 0.91
`cpu_math500`	CPU µVM	MATH-500	competition math	smoke, eval 0.83
`cpu_deepscaler`	CPU µVM	DeepScaleR	scaled math reasoning	smoke, eval 0.70
`cpu_humaneval`	CPU µVM	HumanEval	function-synthesis coding	smoke, eval 0.81
`cpu_gretelsql`	CPU µVM	Gretel SQL	text-to-SQL analytics	smoke, eval 0.65
`cpu_hotpotqa`	CPU µVM	HotpotQA	multi-hop RAG / search	smoke, eval 0.21
`cpu_xlam`	CPU µVM	xLAM-60k	function-calling agent	smoke, eval 1.00
`cpu_mcp`	CPU µVM	ToolACE (MCP)	MCP tool-schema agent	smoke, eval 1.00

Demos (3/3) — scale + substrate / design variants

Demo	Substrate	Task	Real use case	Status
`cpu_harness_scale`	CPU µVM	DeepScaleR (fixed)	scale the rollout driver	offline sweep
`cpu_mixtask`	CPU µVM	math+SQL+CTF+tool	generalist multi-family stream	remote smoke green
`cpu_branching`	CPU CLI Runtime	Claude Code CLI	real-CLI subagents / branching	recipe (fan-out blocker)
`cpu_imagepool`	CPU µVM ×3	datasci (pandas/scipy/sklearn)	capability-routed image pool	recipe (data staged)
`cpu_customruntime`	CPU µVM	trivial	managed vs custom Runtime compare	capability compare
`gpu_customruntime`	GPU inst.	trivial	GPU only via custom Runtime	capability compare
`gpu_local`	local 2×GPU farm	KernelBench	whole-GPU-per-session farm	recipe only

Section 6

6. Special demos — scaling & sandbox variants

Episode shape decides what you scale

The task's shape — not the domain — sets the config. **One microVM serves many families**: the sandbox is task-agnostic (`write_file` / `exec` / `read_file`); task specificity is *files seeded per episode*, not a new image.

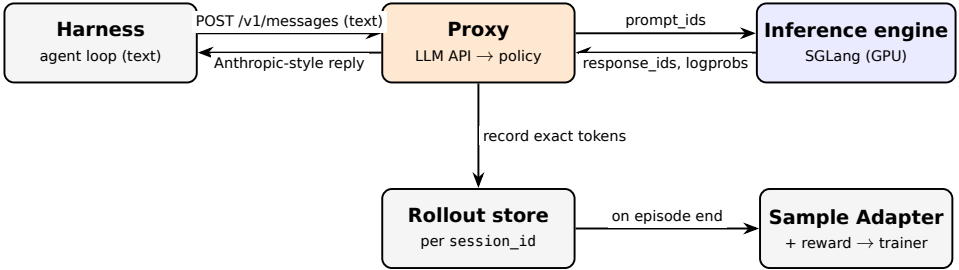
Regime	Example	Bottleneck	Scale
single-turn, long trace	math, reasoning	**generation**	inference
few-turn, heavy checker	KernelBench (CUDA)	**sandbox**	sandbox / GPU
multi-turn, big observations	SWE, RAG, web	context growth	both + warm session

Episode shape decides what you scale (cont.)

- **`--rollout-max-response-len`** — per-turn generation budget; set from the family's p95 per-turn response (math 1k–32k; code 100–800; CUDA kernel 200–1k; tool call 50–300).
- **`EPISODE_MAX_TURNS`** (default 3) — the single biggest lever on episode cost: single-turn → 1; iterative code/SQL → 3–5; long-horizon SWE/web → 15–50+.

Mechanism spotlight — TITO & why the proxy exists

A harness works in **text**; RL trains on **token IDs**. Re-tokenizing decoded text can drift from what the model actually sampled — corrupting the action, the importance ratio, and the loss mask.



Token-in-token-out (TITO): the proxy records the **exact sampled `token_ids` + logprobs** at generation time and forwards them intact, so the trainer learns from the tokens that were really sampled.

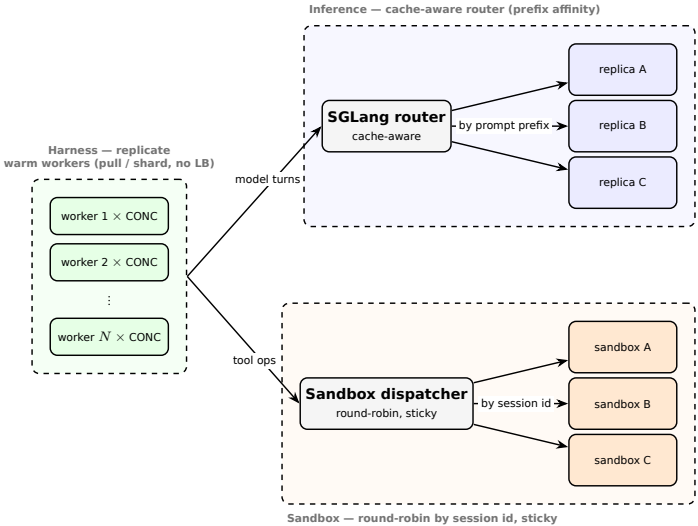
- The proxy is the only component straddling both sides (text ↔ tokens), so it is the only correct place to capture them.
- For binary RLVR rewards this token-level exactness is not optional — slop silently corrupts the signal.

The scaling demo (1/2) — the concrete setup & the three tiers

On the 8×L40S box the demos **fix trainer, inference, and harness** and change only the sandbox ARN + task — the reference-architecture disaggregation made concrete:

Component	Where	Config (shared)
Trainer	4 GPUs	Megatron, GRPO, `global_batch_size=32`, `ACTOR_GPUS=4`
Inference	4 GPUs	SGLang, 2 engines, one **cache-aware router**, `ROLLOUT_GPUS=4`
Harness	in-process (CPU)	`run_episode`, fully-async `MODE=B`, `CONC=4`, `EPISODE_MAX_TURNS=3`
Sandbox	**remote AgentCore**	CPU microVM (arm64) **or** GPU g6 (amd64); one session/episode

The scaling demo (1/2) — the concrete setup & the three tiers (cont.)



The scaling demo (1/2) — the concrete setup & the three tiers (cont.)

*Each tier balances load by a **different rule** — and none needs a load balancer you build.*

The scaling demo (2/2) — the min() law & cpu_harness_scale

The min() law — `throughput = min(inference, sandbox, harness).` Scale two and the bottleneck moves to the third.

- **Diagnose first:** compare `perf/sglang_gen_ms_*` vs `perf/sandbox_ms_*`. Generation dominates → add SGLang replicas; sandbox dominates → add sessions / ARNs / the GPU farm.
- **`cpu_harness_scale`** targets the **easily-forgotten third dial**: the warm in-process harness (GIL, `to_thread` pool, FDs). Scale it by **replicating warm workers** (`N × S`), not by making it ad-hoc. **Problem targeted:** rollout driver becomes the bottleneck once inference and sandbox are both fast.
- **Load-balancing differs by side:** inference → target slime's cache-aware router (prefix affinity); sandbox → the **session id *is* the balancer** (sticky round-robin across a pool of ARNs). You never stand up your own load balancer.

Special demo — `cpu_mixtask`: the generalist stream

Real use case: post-training **one generalist policy** across many skills at once — the frontier norm (Qwen3 [3], Nemotron [4] mix math + code + tool-use + RAG every batch), not one model per task.

Motivation / problem targeted: a single-task demo can't show whether the sandbox + reward path stays correct when the batch is **heterogeneous** — different family, checker, and seed files per row.

- One rollout batch **spans families** (math + SQL + CTF + tool-use) — no harness or sandbox change; each row carries its own family + hidden checker.
- Exercises the claim that **one generic microVM serves many tasks** — task specificity is *seeded files*, not a new image.
- **Status:** remote `NR=3` smoke green.

Special demo — gpu_shared: packing episodes onto one GPU

Real use case: GPU-in-the-loop RL — a **CUDA-kernel agent** (KernelBench [7]) whose checker *compiles and runs* on a real GPU, trained when the GPU fleet is scarce or costly.

Motivation / problem targeted: KernelBench is sandbox-bound, and the isolated path needs *N whole g6 instances at once* — which stalls on a **capacity-starved** fleet (and costs N×).

- All episodes invoke **one** GPU Runtime under a **shared `runtimeSessionId`**; inside, `_handle_farm` leases **one fractional-GPU Ray actor per episode** (keyed by a per-episode `sub_session`).
- `num_gpus=0.5` → two warm-CUDA actors share the GPU; `0.25` → four. **No MIG on L4**, so isolation is software-only (per-process memory caps + an `nvcc flock`).
- **The trade:** isolated = `N` clean instances (unobtainable when the fleet is starved) vs shared = one instance, weaker isolation. **Primary motivation is capacity, not just cost.**
- **Status:** runs **end-to-end on hardware** (`NR=3` smoke, `Job succeeded` + ckpt) via the local-GPU fallback; fractional packing verified, no convergence curve yet. `gpu_local` is the same farm on reserved local GPUs at fraction=1.0 (one whole GPU per session).

Special demo — `cpu_imagepool`: capability-routed image pool

Real use case: a **data-science / analytics agent** whose tasks each pull a different heavy toolkit (pandas, scipy, sklearn) — the reward comes from actually running that stack.

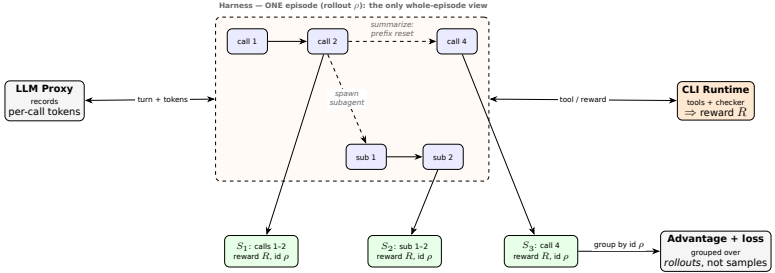
Motivation / problem targeted: those dependency sets are **heavy and incompatible** — they don't all fit under the ~2 GB image cap, and shouldn't co-install in one generic image.

- **One prebuilt image per capability**, each its own Runtime ARN (datasci→pandas / scipy→scipy / sklearn→sklearn); the harness **routes each episode to the image its family needs**.
- Shows the third rung of the dependency ladder: when *files-in-a-session* and *CMD-time install* aren't enough, add a **per-capability ARN** and round-robin the pool.
- **Status:** deployed as VPC Runtimes (3-image ARN map); recipe with data staged.

Special demo — cpu_branching: the real Claude Code CLI as harness

Real use case: training a **real production coding agent** (Claude Code CLI, with subagents + summarization) with RL — not a toy in-process loop. This is where AgentCore hosts the *harness*, not just the sandbox.

Motivation / problem targeted: a real agent **fans one episode out into many samples** (spawns subagents, summarizes → resets the token prefix). Scored naively per-sample, that fan-out silently reweights the GRPO baseline and the loss [1].



Special demo — cpu_branching: the real Claude Code CLI as harness (cont.)

Solution: the CLI runs **inside a CLI-capable CPU Runtime** (image bakes node22 + claude); one episode returns a list[Sample], all tagged with the shared rollout_id ρ and the episode's terminal reward R. **Advantage baseline + loss are normalized per rollout, not per sample** (token-mean loss), so an incidental fan-out can't reweight the update. MODE=A, dp=1 (TP2×PP2), sync save.

Status: recipe only — live blocker (in-sandbox CLI exits before the first model call → fan_out=0); the empty-group crash is fixed.

Special demo — cpu_quiesce: heartbeat keeps the sandbox warm

Real use case: any long RL run — a warm, stateful session (installed deps, seeded repo) must survive the periodic weight-sync pause instead of cold-starting every round.

Motivation / problem targeted: during **weight sync** generation stalls; an idle session is **reclaimed (~15 min)** [D3] and the next round pays a **cold start** — repeatedly.

- A `SessionKeeper` pings the session on a timer, **resetting the idle clock**, so the *same* warm microVM survives the sync pause — no cold start per round.
- Lives **outside the harness** (`KeptSandbox` heartbeat), because the stall is a property of the training loop, not the episode.
- Complements `gpu_snapshot` (recover a *lost* GPU instance via an app-level S3 snapshot): **heartbeat prevents loss; snapshot recovers from it.**

Summary

- **Can:** isolated per-session sandbox for untrusted output (CPU default), fully in-account (IAM/SigV4, VPC), long sessions, bring-your-own container, **GPU on Instances**; a one-hook slime integration with the harness in-process.
- **Cannot:** a microVM has no GPU; no cross-session GPU fractioning (no MIG on L4); a session can't exceed 8 h without moving to Instances (14 d).
- **How to run:** build a sandbox image → deploy a Runtime (+ GPU capacity provider) → seed a task → `invoke_agent_runtime` per tool call → in-sandbox checker gives the reward → slime trains via one `generate` hook. **CPU↔GPU is one env var.**

One image, N sessions, one hook: AgentCore is the isolated per-episode sandbox; slime does the learning.

Section 7

References

References — papers, models & tech reports

- **[1] Agent Lightning** — harnessed agentic RL ·https://arxiv.org/abs/2508.03680
- **[2] DeepSeek-R1** ·https://arxiv.org/abs/2501.12948
- **[3] Qwen3 Technical Report** ·https://arxiv.org/abs/2505.09388
- **[4] Llama-Nemotron** ·https://arxiv.org/abs/2505.00949
- **[5] Kimi K2** ·https://arxiv.org/abs/2507.20534
- **[7] KernelBench** ·https://huggingface.co/datasets/ScalingIntelligence/KernelBench
- **[8] MBPP** ·https://arxiv.org/abs/2108.07732
- **[9] slime** — the trainer ·https://github.com/THUDM/slime ·docs https://thudm.github.io/slime/
- **[27] SGLang / RadixAttention** ·https://arxiv.org/abs/2312.07104 ·cache-aware router https://lmsys.org/blog/2024-12-04-sglang-v0-4/

References — RL algorithms & systems (further reading)

- **[13] GRPO (DeepSeekMath)** ·https://arxiv.org/abs/2402.03300
- **[14] DAPO** ·https://arxiv.org/abs/2503.14476
- **[15] GSPO** ·https://arxiv.org/abs/2507.18071
- **[18] AReaL** — asynchronous RL system ·https://arxiv.org/abs/2505.24298
- **[19] HybridFlow / veRL** ·https://arxiv.org/abs/2409.19256
- **[22] Truncated Importance Sampling** — train/inference mismatch · https://fengyao.notion.site/off-policy-rl
- **[28] APRIL** — active partial rollouts ·https://arxiv.org/abs/2509.18521

References — AgentCore docs, SDK & CLI

- **[D2] Host with Runtime** — isolation, compute types · https://docs.aws.amazon.com/bedrock-agentcore/latest/devguide/agents-tools-runtime.html
- **[D3] Runtime — how it works** — idle 15 m / max 8 h · https://docs.aws.amazon.com/bedrock-agentcore/latest/devguide/runtime-how-it-works.html
- **[D5] Instances (GPU)** · https://docs.aws.amazon.com/bedrock-agentcore/latest/devguide/runtime-instances-how-it-works.html
- **[D8] InvokeAgentRuntime** · https://docs.aws.amazon.com/bedrock-agentcore/latest/APIReference/API_InvokeAgentRuntime.html
- **[D9] CreateAgentRuntime** ·https://docs.aws.amazon.com/bedrock-agentcore-control/latest/APIReference/API_CreateAgentRuntime.html
- **[D13] Observability** — vCPU / GB-hour metrics · https://docs.aws.amazon.com/bedrock-agentcore/latest/devguide/observability.html
- **[D15] Python SDK** — `@app.entrypoint` ·https://github.com/aws/bedrock-agentcore-sdk-python
- **[D16] AgentCore CLI** — `@aws/agentcore` ·https://www.npmjs.com/package/@aws/agentcore
- **[D17] Runtime service contract** — :8080, /invocations, /ping · https://docs.aws.amazon.com/bedrock-agentcore/latest/devguide/runtime-service-contract.html